

Complejidad Computacional

Guía práctica N°2 – @valnrms

Fecha de compilación: 16/04/2025.

Ejercicio 1.

Probar que los siguientes lenguajes están en P.

1.a $\text{COPRIME} = \{\langle a, b \rangle : (a : b) = 1, \text{ es decir, } a \text{ y } b \text{ son coprimos}\}$

(def.) Dos números a, b son coprimos $\iff$ su máximo común divisor es 1.

A su vez, decimos que un **problema** (a.k.a. *lenguaje*) $\mathcal{L}$ pertenece a la clase de complejidad P si existe una máquina determinística $M = (Q, \Sigma, \delta)$ que computa $\chi_{\mathcal{L}}^1$ en tiempo polinomial.

Para el caso de COPRIME, podemos usar el algoritmo de Euclides para calcular el máximo común divisor de a y b .

```
1 def mcd(a, b):
2     while b != 0:
3         a, b = b, a % b
4     return a
```

Donde $a \% b$ es el resto de la división entera de a entre b , que sabemos que es polinomial.

Luego, podemos pensar en el programa que, dado dos números a y b , llama a la función `mcd(a, b)` y verifica si el resultado es 1.

Usando la tesis fuerte de Church-Turing, sabemos que hay una máquina de Turing que puede simular el algoritmo de Euclides que hicimos en Python en tiempo polinomial, y nadie duraría que el programa que chequea si el `mcd(a, b)` es 1 también puede ser simulado por una máquina de Turing en tiempo polinomial.

Por lo tanto, podemos concluir que el lenguaje COPRIME está en P.

1.b $\text{POWER} = \{\langle a, e, b \rangle : a^e = b\}$

Queremos probar que el problema POWER está en P.

Planteo el siguiente programa en Python:

```
1 def power(a, e, b):
2     for i in range(0, e):
3         a = karatsuba(a, a)
4     return a == b
```

Donde `karatsuba(a, a)` es el producto de a por sí mismo usando el algoritmo de Karatsuba mencionado en clase que, dado dos números x y y , calcula su producto en tiempo $O(n^{\log_2 3})$ (i.e., polinomial).

Luego, usando la tesis fuerte de Church-Turing, sabemos que hay una máquina de Turing que puede simular el algoritmo de Karatsuba que hicimos en Python en tiempo polinomial.

Por lo tanto, podemos concluir que el lenguaje POWER está en P.

1.c $\text{TREE} = \{\langle G \rangle : G \text{ es un grafo conexo sin ciclos}\}$

Queremos ver que el problema TREE está en P.

Le corro BFS/DFS desde cualquier nodo y me fijo si en algún momento visité un nodo dos veces o si no visité

¹La función característica. También llamado predicado característico.

todos los nodos.

Si visité un nodo dos veces, el grafo tiene un ciclo. Si no visité todos los nodos, el grafo no es conexo.

Luego, la complejidad de BFS/DFS es $O(|V| + |E|)$, donde $|V|$ es la cantidad de nodos y $|E|$ la cantidad de aristas. Podemos acotarlo como $O(|V|^2)$, ya que en el peor caso cada nodo se conecta a todos los demás nodos.

Por lo tanto, podemos concluir que el lenguaje TREE está en P.

1.d $\mathcal{L}$ donde $|\mathcal{L}| < \infty$ (es decir, probar que todo lenguaje finito está en P)

Por la materia *Lenguajes Formales, Autómatas y Computabilidad*, sabemos que todo lenguaje finito es regular. Por lo tanto, existe un autómata finito determinístico que lo reconoce. Trivialmente se puede extender a una máquina de Turing determinística que lo acepte.

Luego, por la misma materia, sabemos que aceptar/rechazar una cadena en un autómata finito determinístico es un problema en tiempo polinomial (lineal en el tamaño de la entrada).

Por lo tanto, podemos concluir que todo lenguaje $\mathcal{L}$ finito está en P.

Ejercicio 2.

Probar que la clase P está cerrada por unión, intersección y complemento.

2.a Unión

Sean $\mathcal{L}_1, \mathcal{L}_2$ dos lenguajes en P.

Queremos probar que $\mathcal{L}_1 \cup \mathcal{L}_2$ también está en P.

Como $\mathcal{L}_1, \mathcal{L}_2$ están en P, existen máquinas de Turing determinísticas $M_{\mathcal{L}_1}, M_{\mathcal{L}_2}$ que aceptan/rechazan $\mathcal{L}_1, \mathcal{L}_2$ respectivamente en tiempo polinomial.

Luego, hago el siguiente programa en Python que computa $\mathcal{L}_1 \cup \mathcal{L}_2$:

```
1 def unión(x):
2     return M_L_1(x) or M_L_2(x)
```

Donde $M_L_1(x)$ y $M_L_2(x)$ son la implementación en Python de las máquinas de Turing que aceptan $\mathcal{L}_1$ y $\mathcal{L}_2$ respectivamente.

Luego, la complejidad de $M_L_1(x) \text{ or } M_L_2(x)$ es la suma de lo que tarde cada una de las máquinas de Turing, que son ambas polinomiales, luego, la suma de polinomios es un polinomio.

Por lo tanto, podemos concluir que $\mathcal{L}_1 \cup \mathcal{L}_2$ está en P.

2.b Intersección

Idem arriba pero con

```
1 def intersección(x):
2     return M_L_1(x) and M_L_2(x)
```

Agregar todo el texto de arriba es *boilerplate*, se entiende que es lo mismo. Hacer un **and** es comparar dos bits.

Por lo tanto, podemos concluir que $\mathcal{L}_1 \cap \mathcal{L}_2$ está en P.

2.c Complemento

Queremos probar que $\mathcal{L}^c$ está en P, donde $\mathcal{L}$ es un lenguaje en P.

```
1 def complemento(x):
2     return not M_L(x)
```

Hacer un **not** es invertir un bit.

Por lo tanto, podemos concluir que $\mathcal{L}^c$ está en P.

Ejercicio 3.

Probar que los siguientes lenguajes están en NP.

Recordemos la definición de que un problema está en NP:

$\mathcal{L}$ está en NP $\iff \exists p : \mathbb{N} \rightarrow \mathbb{N}, M$ una MT determinística tal que M corre en tiempo polinomial y para todo x :

$$x \in \mathcal{L} \iff \exists u \in \{0, 1\}^{p(|x|)} \text{ tal que } M(\langle x, u \rangle) = 1$$

Intuitivamente, la idea de NP es que, dada una solución al problema, podemos verificar si es correcta en tiempo polinomial.

3.a HAMPATH = $\{\langle G, s, t \rangle :$

G es un grafo con dos nodos s, t tales que hay un camino hamiltoniano de $s \rightarrow t$

Para probar que este problema está en NP podemos hacer lo siguiente.

Digo que un buen certificado es el camino hamiltoniano.

Puedo representarlo como una lista de pares ordenados, donde cada par ordenado (s, t) representa la arista $s \rightarrow t$ del grafo.

Veámos la longitud del certificado. Yo digo que es polinomial, pues a lo sumo hay $O(|V|^2)$ aristas, y cada una ocupa $O(\log|V|)$ bits para ser representada. Nótese que es despreciable el costo de la codificación de los pares ordenados.

Luego, la longitud del certificado es $O(|V|^2 \cdot \log|V|)$, que lo puedo acotar con $O(|V|^3)$.

Ahora, para la máquina que verifica el certificado, lo que hago es:

```
1 def hamiltoniano(G, s, t, Cert):
2     for v,w in Cert
3         if (v,w) not in G:
4             return False
5     return Cert[0] == s and Cert[-1] == t
```

Chequear que cada arista en mi Cert está en el grafo G es agarrar cada v, w y chequear si $G[v][w] == 1$, por lo que la complejidad es $O(|V|^2)$.

Luego, chequear que el primer elemento de Cert es s y el último es t es barato (puedo chequear bit a bit de ambos resultados).

Por lo tanto, la complejidad de `hamiltoniano(G, s, t, Cert)` es polinomial.

Usando la tesis fuerte de Church-Turing, sabemos que hay una máquina de Turing que puede simular el algoritmo que hicimos en Python en tiempo polinomial.

Por lo tanto, podemos concluir que el lenguaje HAMPATH está en NP.

3.b k-CLIQUE = $\{\langle G \rangle : G \text{ es un grafo con subgrafo completo de tamaño mayor o igual a } k\}$

¿No es igual al de abajo?, evidentemente no porque

1. Es otro ítem.
2. No puedo acceder a k pues no está en la codificación.

¿Cómo lo resuelvo?, a consultar.

3.c CLIQUE = $\{\langle G, k \rangle : G \text{ es un grafo con subgrafo completo de tamaño mayor o igual a } k\}$

Digo que un buen certificado es el subconjunto de nodos que forman el subgrafo completo.

Puedo representarlo como una **bitmask** o una lista de vértices.

Nótese que mi certificado tiene longitud polinomial, pues:

- Si es una **bitmask**, tengo $|V|$ bits, donde cada bit representa si el nodo está o no en el subconjunto. Luego me queda $O(|V|)$.
- Si es una lista de vértices, cada entero ocupa a lo sumo $\lceil \log_2 |V| \rceil$ bits para ser representado y tengo $|V|$ números a lo sumo. Luego me queda $O(|V| \cdot \log |V|)$ que lo puedo acotar con $O(|V|^2)$.

Ahora, para la máquina que verifica el certificado, lo que hago es:

```
1 def clique(G, k, Cert):
2     for v,w in Cert
3         if (v,w) not in G:
4             return False
5     return len(Cert) >= k
```

Donde Cert es el certificado que me dan y G es el grafo representado como una matriz de adyacencia con V vértices y E aristas.

Luego, la complejidad de `clique(G, k, Cert)` es $O(|V|^2)$. Usando la tesis fuerte de Church-Turing, sabemos que hay una máquina de Turing que puede simular el algoritmo que hicimos en Python en tiempo polinomial.

Por lo tanto, podemos concluir que el lenguaje CLIQUE está en NP.

3.d k-COLORING = $\{\langle G \rangle : G \text{ es un grafo que se puede particionar en } k \text{ conjuntos independientes}\}$

Mismo problema que el 3.b. No puedo acceder al k .

3.e ISOMORPHISM = $\{\langle G, H \rangle : G, H \text{ son dos grafos isomorfos}\}$

3.f SUBGRAPH ISOMORPHISM = $\{\langle G, H \rangle :$

$G \text{ es un grafo y } H \text{ es isomorfo a un subgrafo inducido de } G\}$

3.g $\neg$ SAT = $\{\langle \varphi \rangle : \neg \varphi \text{ es satisfacible}\}$

Tomo como certificado una representación de la valuación que hace satisfacible Cada proposición que forma parte de una fórmula booleana puede ser asociada a un número entero $p_1, p_2, \dots, p_n$.

Por lo tanto, puedo pensar en un certificado como una bitmask. Puedo acotar su longitud: es menor o igual a la de la codificación de la fórmula.

Luego, la longitud del certificado es $O(|\langle \varphi \rangle|)$. Cada índice prendido de la bitmask representa que el predicado p_i es verdadero.

Luego, la máquina que verifica el certificado hace lo siguiente:

1. Recibe la fórmula booleana y la valuación (certificado).
2. Recorre de izquierda a derecha la fórmula booleana, tomando como la función de valuación al certificado.
 - Notar que esto es lineal respecto al tamaño de la fórmula booleana.
3. Una vez terminado, si el resultado de la valuación de φ nos da falso, entonces devolvemos **True**, sino devolvemos **False**.
4. La complejidad de la máquina es $O(|\langle \varphi \rangle|)$, que es polinomial respecto al tamaño de la entrada.

Por lo tanto, podemos concluir que el lenguaje $\neg$ SAT está en NP.

Ejercicio 4.

Probar que los siguientes problemas están en coNP.

4.a $\text{PRIME} = \{\langle n \rangle : n \text{ es primo}\}$

4.b $\text{GIRTH} = \{\langle G, k \rangle : G \text{ es un grafo tal que todos sus ciclos simples tienen } k \text{ o menos vértices}\}$

4.c $\text{TAUTOLOGY} = \{\langle \varphi \rangle : \varphi \text{ es una tautología}\}$

Ejercicio 5.

Considerar los siguientes lenguajes:

- $\text{PATH} = \langle G, s, t, k \rangle :$

G es un digrafo con dos nodos s, t tales que hay un recorrido de longitud menor o igual a k de $s \rightarrow t$

- $\text{EVEN-PATH} = \langle G, s, t, k \rangle :$

G es un digrafo con dos nodos s, t tales que hay un recorrido de longitud par y menor o igual a k de $s \rightarrow t$

Dado un digrafo G , definimos G' como el digrafo que tiene dos copias $(v, p)^2$ y $(v, i)^3$ de cada vértice $v \in V(G)$ donde $(v, x) \rightarrow (w, y)$ es una arista de G' si y solo si $v \rightarrow w \in E(G)$ y $x \neq y$.

5.a Demostrar que $\langle G, s, t, k \rangle \in \text{EVEN-PATH} \iff \langle G', (s, p), (t, p), k \rangle \in \text{PATH}$.

$\Rightarrow$) Como $\langle G, s, t, k \rangle \in \text{EVEN-PATH}$ sé que existe camino con longitud n par tal que

$$\underbrace{s \rightarrow q_1 \rightarrow \dots \rightarrow q_{n-1} \rightarrow t}_{n \leq k}$$

Estoy pensando en $s = q_0 \rightarrow q_1 \rightarrow \dots \rightarrow q_{n-1} \rightarrow q_n = t$.

Ahora, debido a cómo se construye el grafo G' , sé que va a pivotar entre pares e impares, es decir,

$(q_j, p) \rightarrow (q_{j+1}, i)$ y $(q_{j'}, i) \rightarrow (q_{j'+1}, p)$, donde j es par y j' impar.

Luego, como la longitud del camino es par, el camino se puede ver de la siguiente manera:

$$\underbrace{(s, p) \rightarrow (q_1, i) \rightarrow (q_2, p) \rightarrow \dots \rightarrow (q_{n-2}, i) \rightarrow (q_{n-1}, p) \rightarrow (t, p)}_{n \leq k}$$

Por lo tanto, existe un camino de longitud n par tal que $(s, p) \rightarrow (q_1, i) \rightarrow (q_2, p) \rightarrow \dots \rightarrow (q_{n-2}, i) \rightarrow (q_{n-1}, p) \rightarrow (t, p)$ en G' . Por hipótesis $n \leq k$, por lo que cumple.

$\Leftarrow$) Asumo que $\langle G', (s, p), (t, p), k \rangle \in \text{PATH}$, es decir, existe un camino de longitud $n \leq k$ tal que $(s, p) \rightarrow (q_1, i) \rightarrow \dots \rightarrow (q_{n-1}, p) \rightarrow (t, p)$ en G' .

Nótese que como invariante, si estoy parado en (q_j, p) , es porque j es par, y si estoy parado en $(q_{j'}, i)$, es porque j' es impar.

Luego, sabiendo que $s = q_0$ y $t = q_n$, sé que el camino de longitud n es par.

Por lo tanto, existe un camino de longitud n par tal que $s \rightarrow q_1 \rightarrow \dots \rightarrow q_{n-1} \rightarrow t$ en G .

5.b Mostrar que la reducción de EVEN-PATH a PATH implicada por el punto anterior es polinomial.

Me gusta cómo plantearon este ejercicio, te lleva de la manito para probar reducciones. Banco.

Se dice que un lenguaje $\mathcal{L}$ es reducible a otro lenguaje $\mathcal{L}'$ si existe una función **computable en tiempo polinomial** f tal que $x \in \mathcal{L} \iff f(x) \in \mathcal{L}'$, para todo x .

Defino la función f de la siguiente manera:

² p de par.

³ i de impar.

$$f(\langle G, s, t, k \rangle) = \langle G', (s, p), (t, p), k \rangle$$

Y sé que esta reducción vale el $\iff$ por la demostración anterior. Es decir:

$$\langle G, s, t, k \rangle \in \text{EVEN-PATH} \iff \langle G', (s, p), (t, p), k \rangle \in \text{PATH}$$

es verdadero.

Falta probar que f es computable en tiempo polinomial.

1. Hago una copia de todos los vértices de G y los guardo en G' , tanto con el sufijo p como con el sufijo i .
 - Me toma algo así como $O(2 \cdot |V|)$.
2. Agarro todas las aristas de G y las guardo en G' de la siguiente manera:
 - Si tengo una arista (u, v) , guardo $(u, p) \rightarrow (v, i)$ y $(u, i) \rightarrow (v, p)$ en G' .
 - Me toma algo así como $O(2 \cdot |E|)$.

Luego, la complejidad de f es $O(|V| + |E|)$, que es polinomial.

Ejercicio 6.

Considerar los siguientes lenguajes:

- 2-PARTITION = $\{\langle X \rangle :$
 X es un subconjunto finito de los números naturales que se puede particionar en dos conjuntos X_1, X_2 tales que $X_1 \cap X_2 = \emptyset, X_1 \cup X_2 = X$ y $\sum_{x \in X_1} x = \sum_{x \in X_2} x\}$
- 3-PARTITION = $\{\langle X, t \rangle : X$ es un subconjunto finito de los números naturales tal que $\sum_{x \in X} x = \frac{|X|t}{3}, \max_{x \in X} x < \lfloor \frac{t}{2} \rfloor$ y X se puede particionar en $\frac{|X|}{3}$ triplas donde cada una suma $t\}$
- RECTANGLE-PACKING = $\{\langle R, r_1, \dots, r_k \rangle : R$ es un rectángulo que se puede cubrir completamente y sin superposición usando los rectángulos $r_1, \dots, r_k$ con traslaciones y/o rotaciones⁴

6.a Dada una instancia $\langle X \rangle$ de 2-PARTITION, se define la instancia $\langle R, r_1, \dots, r_{|X|} \rangle$ de

RECTANGLE-PACKING donde R tiene base $\sum_{x \in X} \frac{x}{2} + 4$ y altura 2, y r_i es un rectángulo de base $x_i + 2$ y altura 1, para cada $1 \leq i \leq |X|$. Demostrar que $\langle X \rangle \in \text{2-PARTITION} \iff \langle R, r_1, \dots, r_{|X|} \rangle \in \text{RECTANGLE-PACKING}$.

6.b Dada una instancia $\langle X, t \rangle$ de 3-PARTITION, se define la instancia $\langle R, r_1, \dots, r_{|X|} \rangle$ de

RECTANGLE-PACKING donde R tiene base $\frac{|X|}{3}$ y altura $t + |X|$ y r_i es un rectángulo de base 1 y altura $\frac{|X|}{3} + x_i$, para cada $1 \leq i \leq |X|$. Demostrar que $\langle X, t \rangle \in \text{3-PARTITION} \iff \langle R, r_1, \dots, r_{|X|} \rangle \in \text{RECTANGLE-PACKING}$.

6.c Mostrar que las reducciones implicadas por los puntos anteriores son polinomiales en función de los tamaños de las entradas.

Ejercicio 7.

Explicar por qué la identidad no es una reducción polinomial de un lenguaje Π a Π^c . Concluir que las nociones de NP y coNP son altamente sensibles a la "etiqueta" de la respuesta.

Es trivial que no vale. Supongamos que vale, entonces es cierto que para todo x ,

$$x \in \Pi \iff x \in \Pi^c.$$

Absurdo, pues Π^c es el complemento de Π .

⁴Un rectángulo en este contexto se representa con dos números naturales indicando su ancho y alto.

Concluimos que la noción de NP y coNP son altamente sensibles a la "etiqueta"⁵ de la respuesta.

Ejercicio 8. Considerar el siguiente lenguaje:

- $\text{CONNECTED} = \langle G, s, t \rangle : G \text{ es un digrafo y } s, t \text{ son dos nodos de } G \text{ tales que hay un recorrido de } s \rightarrow t \}$

Para un digrafo G , sea H el digrafo que tiene un vértice (S, v) para cada $S \subseteq V(G)$ y cada $v \in V(G)$, donde $(S, v) \rightarrow (R, w)$ es una arista de H si y solo si $w \notin S$, $R = S \cup \{w\}$ y $v \rightarrow w$ es una arista de G .

8.a Demostrar que $\langle G, s, t \rangle \in \text{HAMPATH} \iff \langle H, (\{s\}, s), (V(G), t) \rangle \in \text{CONNECTED}$.

$\Rightarrow$) Asumo que $\langle G, s, t \rangle \in \text{HAMPATH}$, es decir, existe un camino de longitud n tal que $s = q_0 \rightarrow q_1 \rightarrow \dots \rightarrow q_{n-1} \rightarrow t$.

Luego, en H , existe una arista $(\{s\}, s) \rightarrow (\{s, q_1\}, q_1)$.

Si extendemos el argumento llegamos a que existen aristas $(\{s\}, s) \rightarrow (\{s, q_1\}, q_1) \rightarrow \dots \rightarrow (\{s, q_1, \dots, q_{n-1}\}, q_{n-1}) \rightarrow (\{s, q_1, \dots, q_{n-1}, t\}, t)$ en H .

8.b Mostrar que la reducción de HAMPATH a CONNECTED implicada por el punto anterior **no** es polinomial.

Ejercicio 9.

Sea $\mathcal{L} \in \text{RECURSIVE}$.⁶ Probar que $\mathcal{L} \leq_p \text{HALTING}$.

Ejercicio 10.

Probar que $\text{NP} \subseteq \text{RECURSIVE}$. Concluir que $\text{HALTING} \notin \text{NP}$.

Ejercicio 11.

Considerar el siguiente argumento que busca probar que HALTING pertenece a NP.

Defino una máquina $M(x, c)$ que toma el programa x y un certificado c de longitud 1.

El certificador funciona de la siguiente forma: si $c = 1$, devuelve 1, y si $c = 0$, devuelve 0.

Vale que si x es un programa que para (i.e. $M_{x(x)}$ para), entonces hay un certificado tal que la máquina $M(x, c)$ devuelve 1.

De la misma forma, si la máquina no para hay un certificado tal que la máquina $M(x, c)$ devuelve 0.

Concluimos que HALT está en NP.

— Ayudante anónimo

¿Por qué esto no prueba que HALT está en NP?

⁵Es la forma en que está formulada la decisión del problema: si se espera como salida un "sí" o un "no". En problemas de decisión, esta etiqueta determina qué instancias son aceptadas. Por ejemplo, SAT pregunta si una fórmula es satisfacible (respuesta esperada: "sí"), mientras que UNSAT pregunta si no es satisfacible (respuesta esperada: "sí" también, pero en el complemento). Cambiar la etiqueta puede afectar a qué clase (como NP o coNP) pertenece el problema.

⁶RECURSIVE es el conjunto de problemas decidibles o bien *recursivos*.